תרגול 2
רשתות חברתיות

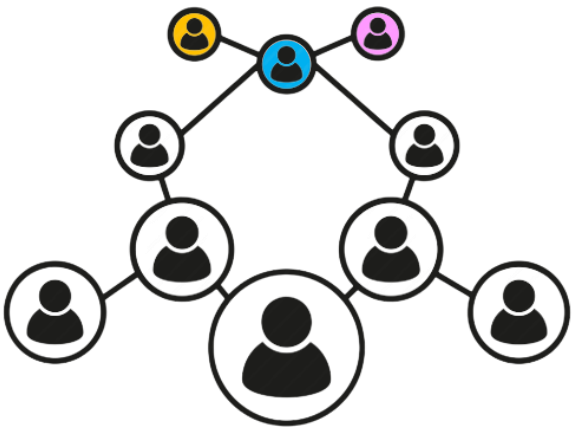
רקע

שיעור 2 היה עמוס מאוד, בעיקר בגלל שלמדנו בו גישת תכנות חדשה – **תכנות מונחה עצמים**.
בגישת OOP (Object Oriented Programming) אנחנו מפצלים את התוכנית שלנו לישויות עצמאיות שלהן אנחנו קוראים **מחלקות**, חלק מהמחלקות מבודדות, וחלק משתמשות במחלקות אחרות.
כל מחלקה עושה תפקיד מסוים, וביחד כל החלקים מרכיבים תוכנה או מערכת שלמה.

אחד היתרונות שיש לגישת OOP הוא שאפשר לבנות מערכות מורכבות בצורה הדרגתית, חלק אחר חלק.
אבל כדי לשלוט בגישת התכנות החדשה צריך לתרגל, לכן לאורך הסמסטר נעבוד על פיה.

מטרה

בתרגיל נבנה רשת חברתית צנועה שבה משתמשים יוכלו להתחבר לפרופיל שלהם באמצעות מכשירים שונים, לכתוב פוסטים, ולהוסיף חברים.



אלה השלבים שנעבור:

למידה עצמית & האופרטור	למידה עצמית מחרוזות (std::string)	כתיבת מחלקת Device	כתיבת מחלקת User	כתיבת מחלקות Profile-ו Page	כתיבת מחלקת Social Network
• למידה עצמית על נושא reference • הגשת תשובות לשאלות תאורטיות	• למידה עצמית על נושא מחרוזות	• כתיבת הגדרות • מימוש המחלקה	• כתיבת הגדרות • מימוש המחלקה • שימוש ברשימה מקושרת	• כתיבת הגדרות • מימוש המחלקות • שימוש ברשימה מקושרת • שלב בונוס	• כתיבת הגדרות • מימוש המחלקה • שימוש ברשימה מקושרת

נתרגל מיומנויות חשובות:



- נתרגל שימוש במשתנים מסוג reference.
- נתנסה בכתיבת מחלקות ושימוש נכון בכימוס (encapsulation).
- נשתמש במחלקה `std::string`.
- נעבוד עם רשימות מקושרות.

את התרגיל צריך להגיש ב-GIT: לינק להוראות שימוש ב-GIT.

כדאי לקרוא גם דגשים לתכנות נכון.

בנוסף ישנן בדיקות אוטומטיות שרצות באתר GitHub,
חשוב לוודא שהקוד שעלה עובר את הבדיקות של השבוע.



"Practice Makes Perfect"

בהצלחה יא אלופות ואלופים!

שלב 1: לימוד עצמי – האופרטור &

שלב 1 האופרטור &	שלב 2 מחרוזות	שלב 3 מחלקת Device	שלב 4 מחלקת User	שלב 5 מחלקות Profile -ו Page	שלב 6 מחלקת Social Network

למידה עצמית

קראו במאמר הבא על המושג reference וענו על השאלות הבאות:

1. מהו reference וביצד מגדירים reference למשתנה?
2. מנו שני יתרונות בהעברת משתנים לפונקציה by reference על פני שיטות אחרות.
3. מה הם ההבדלים בין pointer לreference? מדוע reference נחשב ל"בטוח" יותר?
4. נתונה הפונקציה ונתון משתנה y מסוג int:

```
void square(int x, int& result)
{
	result = x * x;
}
```

האם הקריאות הבאות תקינות? אם לא, הסבירו מה הבעיה.

א. `square(3, y);`
ב. `square(3, &y);`
ג. `square(3, 6);`

5. מה הבעיה בכל אחת מהפונקציות הבאות :

a.

```
int&  getLocalVar()
{
	int x = 10;
	return x;
}
```

b.

```
int&  getDynamicVar()
{
        int *x = new int(10);
        return *x;
}
```

סיכום חלק 1 – עליכם/ן להעלות לגיט את הקבצים הבאים: **answers.pdf**

שלב 2: מחרוזות

למידה עצמית - מחלקת `std::string`

בחלק הזה נכיר מחלקה מאוד שימושית אשר מומשה עבורנו בספריית std - הספרייה הסטנדרטית של C++.

לאורך השנה, וגם בתוכניות שנכתוב בהמשך, נראה שאחד הדברים הנפוצים שחוזרים הוא שימוש במחרוזות = רצף של תווים.

עד עכשיו עבדנו בשפת C עם מערך תווים (`char []`), או עם מצביעים שאליהם הקצינו זיכרון (`char*`). יכול להיות שחלקנו אפילו השתמש בספריות חיצוניות שהכילו פונקציות שימושיות שעובדות עם הסוגים האלו כמו לדוגמא `strcmp()`, `strtok()`, `strcat()`.

עכשיו לאחר שלמדנו על מחלקות ב-cpp, נוכל להכיר מחלקה שימושית אשר נועדה לייצג רצף תווים – `std::string`.

אובייקט מסוג `std::string` נבנה באמצעות רצף תווים.

הנה כמה דוגמאות ליצירת מופע של המחלקה `std::string`

```
std::string my_string1 = "";
std::string my_string2 = "lalalala";
std::string my_string3 = "123 and 345";
std::string my_string4 = "#!@%$^&*()";

const char* c_str = "bla bla";
std::string my_string = c_str;  // mystring = “bla bla"
```

באובייקט של `std::string` קיימים אופרטורים אשר הופכים אותו להיות דומה מאוד למערך התווים אליו אנחנו רגילים, לדוגמא האופרטור [ ] (סוגריים מרובעים) מאפשר לגשת לתו במחרוזת כמו שאנו ניגשים לתא במערך:

```
std::string my_string3 = "123 and 456";
char c1 = my_string3[5]; // c1 = ‘n’
char c2 = my_string3[2]; // c2 = ‘3’
char c3 = my_string3[10]; // c2 = ‘5’
char c4 = my_string3[12]; // run time error!!!
```

```
std::string s1 = "Live";
s1[1] = 'o';
```

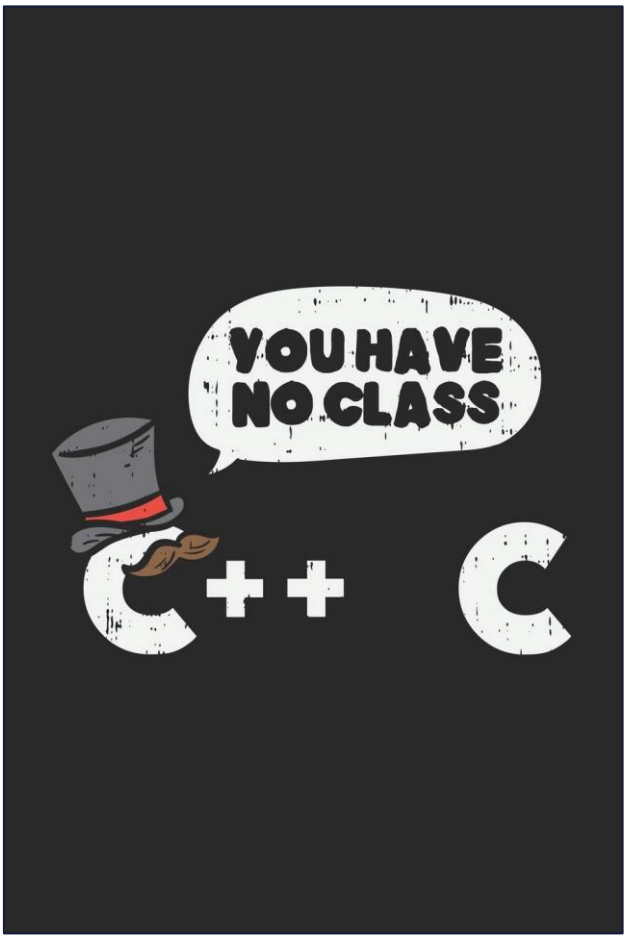
אופרטור שימושי נוסף הוא אופרטור החיבור '+', אשר מאפשר לנו לחבר מחרוזת עם תו, או מחרוזת עם מחרוזת אחרת. ניתן גם לחבר כמה מחרוזות ברצף, העיקר שלפחות אחד מבין שני גורמים שחיברנו הוא אובייקט מסוג `std::string`

```
std::string s1 = "Donald";
std::string s2 = "123";
std::string s3 = s1 + s2; // s3 = “Donald123”

std::string s4 = "Banel";
std::string s5 = "Static";
std::string s6 = s4 + " & " + s5; // s6 = “Banel & Static”
std::string s7 = s5 + " & Banel"; // s7 = “Static & Benel”
std::string s8 = "Banel & " + s5; // s8 = “Banel & Static”

std::string str1 = "Yofi " + "Tofi"; // compilation error!!!
```

המחלקה חושפת החוצה ממשק עשיר עם מתודות מאוד שימושיות, עברו עליו, בהמשך התרגיל הממשק עשוי להקל עליכם/ן.



שלב 3: רשת חברתית – מחלקת Device

שלב 1	שלב 2	שלב 3	שלב 4	שלב 5	שלב 6
האופרטור &	מחרוזות	מחלקת Device	מחלקת User	מחלקות Profile -ו Page	מחלקת Social Network

בתרגיל נבנה רשת חברתית צנועה שבה משתמשים יוכלו להתחבר לפרופיל שלהם באמצעות מכשירים שונים, לכתוב פוסטים, ולהוסיף חברים. אנחנו נתחיל בליצור מחלקה המייצגת מכשיר (`Device`), ממנה נמשיך למחלקה שמייצגת משתמש/ת (`User`), אחר כך נמשיך למחלקת `Profile` ובסוף נבנה את המחלקה `Social Network` שתשתמש בכל המחלקות הקודמות שבנינו.

מחלקת Device – רקע

המשתמשים של הרשת שלנו יוכלו להתחבר לרשת ממכשירים שונים, מכשיר יכול להיות מחשב נייח או לפטופ, טאבלט או טלפון. המחלקה הראשונה שנכתוב תהיה מחלקת `Device` אשר מייצגת מכשיר שמתחבר לרשת שלנו .

מכשיר - Device (המשך...)	
תיאור	**מתודה/פעולה**
מחזירה את שם מערכת ההפעלה של המכשיר. **הערה:** הערך החוזר מהמתודה הוא מסוג std::string שעליו למדנו בחלק הקודם.	`std::string getOS()`
מחזירה האם המכשיר במצב "פועל" **הערה**: מכשיר חדש מתחיל במצב "פועל"	`bool isActive()`
משנה את מצב המכשיר ל-"פועל"	`void activate()`
משנה את מצב המכשיר ל-"לא פועל"	`void deactivate()`

מחלקת Device

לרשותכם/ן הקובץ *Device.h*, שם תוכלו למצוא ערכים קבועים שהוגדרו מראש כדי שיהיה נוח לבדוק את המחלקה.

- `enum DeviceType` – מגדיר 4 סוגי מכשירים – {מחשב נייח, לפטופ, טאבלט, טלפון } .
- הגדרות (`#define`) לסוגי מערכות ההפעלה שרצות על המכשירים .

אם תרצו תוכלו להוסיף סוגים נוספים, אבל אין למחוק את הסוגים הקיימים.

השלימו את הקובץ *Device.h* : רשמו את חתימות המתודות של המחלקה והצהירו על השדות שלה.
שימו 💙 , בטבלה הבאה רשומות המתודות של המחלקה `Device`, תוכלו באמצעותה גם להסיק בעצמכם/ן מה השדות שהמחלקה צריכה להחזיק.

אחרי שסיימתם/ן את קובץ ההגדרות, **צרו קובץ חדש** בשם *Device.cpp* וממשו את המתודות שהוגדרו בקובץ h.

הנחיות והערות חשובות:

- בהוראות לא רשומות החתימות המדויקות של המתודות של המחלקה, אבל האחריות שלכם/ן היא להוסיף `const` ולקבל פרמטרים כ-reference היכן שצריך.

מכשיר - Device	
תיאור	**מתודה/פעולה**
מאתחלת אובייקט של מכשיר חדש. **הערה:** הפונקציה תיקרא לפני השימוש באובייקט.	`init(unsigned int id, DeviceType type, std::string os)`
מחזירה את המספר המזהה של המכשיר.	`unsigned int getID()`
מחזירה את סוג המכשיר. **הערה**: הערך החוזר מהמתודה הוא מסוג `DeviceType` שהוגדר מראש בקובץ התרגיל.	`DeviceType getType()`

- חשפו החוצה מהמחלקה כמה שפחות מידע: מה שאפשר הגדירו `private`, ומה שנרצה לחשוף החוצה הגדירו `public`.

- רשמו בקובץ ה-H כמה שפחות הגדרות וחתימות.
לדוגמה, לרוב אין צורך לעשות `#include <iostream>` בקובץ ה-H, ניתן לכתוב זאת בקובץ המימוש (cpp).

- זכרו שכדי להשתמש במחלקת `std::string` יש לבצע `#include <string>` .

- מומלץ לבדוק את המחלקה לפני שממשיכים לסעיף הבא, תוכלו להיעזר בקובץ *test1Device.cpp* שניתן יחד עם קבצי התרגיל.

- ⚠️ חשוב! ⚠️
הקפידו לתת שמות נכונים לכל דבר שאתם/ן יוצרים/ות, כלומר ע"פ מה שרשום בהוראות. הטסטים שבעזרתם ייבדק התרגיל משתמשים בשמות האלו והם לא יעברו אם השמות של הקבצים/מחלקות/מתודות שונים מההוראות.

- שימו 💙 , בקובץ *Tester.cpp* יש בדיקות לכל שלבי התרגיל, ואם לא מימשנו את כל התרגיל הקובץ לא יתקמפל. תוכלו להתגבר על הבעיה בשתי דרכים:
  - אל תשתמשו ב-*Tester.cpp* השתמשו בקובץ הבדיקות של חלק 1 – *test1Device.cpp*
  - שימו את כל שאר הקוד בהערה ותעבדו ישירות עם הקובץ *Tester.cpp*.



שלב 4: רשת חברתית – מחלקת User

מחלקת User – רקע

השלב הבא ביצירת הרשת החברתית הוא לייצג משתמש של המערכת.
כדי להירשם לרשת משתמש צריך ת.ז, שם משתמש וגיל.
למשתמש יש רשימת מכשירים שאיתם הוא מתחבר לרשת.

מחלקת User

בחלק הזה ניצור שני קבצים חדשים – *User.h* ו-*User.cpp*

בנוסף, בשלב הזה נשתמש ברשימה מקושרת שנכתבה ומומשה עבורנו בקבצים ***DeviceList.h*** ו-***DeviceList.cpp***.

כתבו את הגדרות המחלקה וממשו אותה ע"פ התיאור.
שימו 💛 , בטבלה הבאה רשומות המתודות של המחלקה `User`, תוכלו באמצעותה גם להסיק בעצמכם/ן מה השדות שהמחלקה צריכה להחזיק.

משתמש - User	
מתודה/פעולה	**תיאור**
`void init(unsigned int id, std::string username, unsigned int age)`	מאתחלת אובייקט של משתמש חדש. **הערה:** הפונקציה תיקרא לפני השימוש באובייקט.
`void clear()`	מנקה את האובייקט ומוחקת זיכרון שהוקצה לאובייקט. **הערה:** אם במהלך יצירת האובייקט איתחלנו אובייקטים אחרים, אז בפונקציה זו אנו ננקה גם אותם. אולם אין צורך להשתמש ב-`delete` אלא אם כן הייתה הקצאת זיכרון דינמית באמצעות המילה `new` .
`unsigned int getID()`	מחזירה את תעודת הזהות של המשתמש.
`std::string getUserName()`	מחזירה את שם המשתמש.
`unsigned int getAge()`	מחזירה את גיל המשתמש.
משתמש - User (המשך...)	
מתודה/פעולה	**תיאור**
`DevicesList& getDevices();`	מחזירה הפניה לרשימת המכשירים של המשתמש. **הערה:** הסוג החוזר מהפונקציה הוא reference שעליו למדנו בתחילת התרגיל. חשבו למה אנחנו רוצים להחזיר reference ולא עותק...
`void addDevice(Device newDevice);`	מוסיפה מכשיר לרשימת המכשירים של המשתמש. **הערה:** הפונקציה צריכה לעשות שימוש במחלקה `DeviceList` שניתנה עם קבצי התרגיל.
`bool checkIfDevicesAreOn()`	מחזירה `true` אם כל המכשירים שבבעלות המשתמש נמצאים במצב "פועל" **הערה:** הפונקציה צריכה לעשות שימוש במחלקה `DeviceList` שניתנה עם קבצי התרגיל.

הנחיות והערות חשובות:

- בהוראות לא רשומות החתימות המדויקות של המתודות של המחלקה, אבל האחריות שלכם/ן היא להוסיף `const` ולקבל פרמטרים כ-reference היכן שצריך.
- זכרו לבצע `#include "DeviceList.h"` כדי שתוכלו להשתמש ברשימה שסופקה לכם/ן.
- שימו 💛 , מומלץ לבדוק את המחלקה לפני שממשיכים לסעיף הבא, תוכלו להיעזר בקובץ *test2User.cpp* שניתן יחד עם קבצי התרגיל.



שלב 5: רשת חברתית – מחלקות Page ו-Profile

מחלקות Page ו-Profile – רקע

עכשיו נוכל להתקדם שלב נוסף, וליצור מחלקה המייצגת פרופיל ברשת החברתית שלנו.
לכל פרופיל יש בעלים – המשתמש (User) שהפרופיל שייך לו/ה. בתוך הפרופיל יש עמוד (Page) שבו אפשר לפרסם סטטוס ופוסטים, וגם רשימת חברים (משתמשים אחרים).

מחלקת Page

נתחיל במחלקה Page, המייצגת עמוד בפרופיל של משתמש מסוים.
העמוד מכיל את הסטטוס של המשתמש ואת הפוסטים שלו.
בחלק הזה ניצור 2 קבצים חדשים - *Page.h, Page.cpp*

בתוך עמוד נחזיק שני שדות מסוג מחרוזת:

- עבור **הסטטוס** שיהיה רשום בראש העמוד.

- עבור **הפוסטים** שכתובים בעמוד.
  פוסט יכול להיות משפט או רצף של מילים שהמשתמש מעוניין לשתף.
  כל הפוסטים מאוחסנים יחד במחרוזת אחת, בכל פעם שמתווסף פוסט לעמוד, נוספת שורה חדשה למחרוזת הפוסטים.

ממשו את המחלקה Page ע"פ התיאור הבא, הקפידו להגדיר את שני השדות שתוארו למעלה:

 **עמוד - Page**	
תיאור	**מתודה/פעולה**
מאתחלת אובייקט של עמוד חדש עם מחרוזת סטטוס ריקה ומחרוזת פוסטים ריקה. **הערה:** הפונקציה תיקרא לפני השימוש באובייקט.	`init()`

מחזירה מחרוזת עם הפוסטים שבעמוד, כל פוסט בשורה חדשה:
לדוגמה:
`Hello world!`
`Good morning :)`
`Magshimim forever ***`
	`std::string getPosts()`

מחזירה את הסטטוס. **הערה**: הסטטוס זה מחרוזת שתמיד מופיעה בראש העמוד.	`std::string getStatus()`

`void clearPage()`	מנקה את הפוסטים מעמוד הפרופיל. **הערה**: המתודה מאפסת את מחרוזת הפוסטים, אבל לא משנה את הסטטוס של המשתמש.
`void setStatus(std::string status)`	משנה את הסטטוס.
`void addLineToPosts(std::string new_line)`	מוסיפה שורה חדשה למחרוזת הפוסטים.

מחלקת Profile

כעת נעבור למחלקה Profile אשר מחזיקה את המידע על המשתמש ברשת החברתית.

בחלק הזה ניצור 2 קבצים חדשים – *Profile.h, Profile.cpp*
בנוסף, בשלב הזה נשתמש ברשימה מקושרת שנכתבה ומומשה עבורנו בקבצים *UserList.h* ו-*UserList.cpp*.

בתוך פרופיל נחזיק שלושה שדות:

- המשתמש/ת (User) שהפרופיל בבעלותו/ה.
- עמוד (Page).
- רשימת חברים (UserList).

הגדירו את המחלקה וממשו אותה ע"פ התיאור הבא:

Profile – פרופיל	
`void init(User owner);`	מאתחלת אובייקט של פרופיל חדש. **הערה:** הפונקציה תיקרא לפני השימוש באובייקט.
`void clear()`	מנקה את האובייקט ומוחקת זיכרון שהוקצה לאובייקט. **הערה:** אם במהלך יצירת האובייקט אתחלנו אובייקטים אחרים, אז בפונקציה זו אנו ננקה גם אותם. אולם אין צורך להשתמש ב-delete אלא אם כן הייתה הקצאת זיכרון דינמית באמצעות המילה new .
`User getOwner()`	מחזירה העתק של אובייקט המשתמש שהפרופיל בבעלותו.
`void setStatus(std::string new_status)`	משנה את הסטטוס שבעמוד של בעל/ת הפרופיל לסטטוס החדש. **הערה**: המתודה משתמשת במתודה Page::setStatus()
`void addPostToProfilePage(std::string post)`	מוסיפה פוסט לעמוד של בעל/ת הפרופיל . **הערה**: המתודה משתמשת במתודה Page::addLineToPage()
`void addFriend(User friend_to_add)`	מוסיפה חבר/ה לרשימת החברים של הפרופיל. **הערה:** הפונקציה צריכה לעשות שימוש במחלקה UserList שניתנה עם קבצי התרגיל.

 **פרופיל – Profile (המשך...)**	

מחזירה מחרוזת עם הסטטוס והפוסטים שבעמוד של בעל/ת הפרופיל.
הפורמט של המחרוזת הוא כזה:
`Status: <User's status>`
`**********`
`**********`
`<post1>`
`<post2>`
`<post3>`
`...`
דוגמה:
`Status: Status: Which witch watched which watch???`
`**********`
`**********`
`Hello world!`
`Good morning :)`
`Magshimim forever ***`
	`std::string getPage();`

מחזירה מחרוזת עם שמות החברים של בעל/ת הפרופיל מופרדים בפסיק.
הפורמט של המחרוזת הוא כזה:
`<friend1>,<friend2>,<friend3>,...,<friendn>`
דוגמה:
`Avi,Nitzan,Rinat,Shlomi`
הערה: הפונקציה צריכה לעשות שימוש במחלקה `UserList` שניתנה עם קבצי התרגיל.
	`std::string getFriends();`

מחזירה מחרוזת עם רשימת החברים ששם המשתמש שלהם הוא באותו האורך כמו שם המשתמש של הבעלים של הפרופיל. לדוגמה: נניח שהשם של הבעלים של הפרופיל הוא `"Shlomo"` ושרשימת החברים היא: `"Avner,Nitzan,Motti,Maya,Lilach"` המתודה תחזיר את המחרוזת הבאה: `"Nitzan,Lilach"`	`std::string getFriendsWithSameNameLength()`

הנחיות והערות חשובות:

- בהוראות לא רשומות החתימות המדויקות של המתודות של המחלקה, אבל האחריות שלכם/ן היא להוסיף `const` ולקבל פרמטרים ב-reference היכן שצריך.
- זכרו לבצע `#include "UserList.h"` כדי שתוכלו להשתמש ברשימה שסופקה לכם/ן.

- המחרוזת שחוזרת מהמתודות `getFriends()` , `getFriendsWithSameNameLength()` לא מכילה פסיק `','` בסוף המחרוזת, חשבו איך לגרום לתו לא להופיע.

- שימו 💚 , מומלץ לבדוק את המחלקה לפני שממשיכים לסעיף הבא,
תוכלו להיעזר בקובץ ***test3Profile.cpp*** שניתן יחד עם קבצי התרגיל.

- ❗ ❗ ❗ <u>הערה בקשר לרשימות המקושרות הניתנות בתרגיל</u> ❗ ❗ ❗

אם תעברו על הקוד שבקבצים שסופקו יחד עם הוראות התרגיל תראו שיש כפל קוד.
כל מחלקה שמייצגת רשימה בתרגיל זהה במעט לחלוטין למחלקות האחרות,
והדבר היחיד ששונה זה סוג האיברים שכל רשימה מחזיקה.

חשוב שנבין שקוד שחוזר על עצמו בצורה כזו הוא לא קוד יעיל, ולאורך הקורס ננסה לא לחזור על עצמנו פעמיים ולא להשתמש ב-copy & paste יותר מידי... אבל כרגע בשלב הזה של הקורס אין לנו כלים לפתור את זה.
בשבוע 9 נלמד על מנגנון שמאפשר להגדיר מחלקות כלליות, שלא חשוב איזה סוג משתנים הן מחזיקות, והקוד ייכתב עבורנו אוטומטית.





החלק הבא אינו חובה, אבל זה תרגול מצוין בעבודה עם מחרוזות.

הועסקתם/ן ע"י ארגון בטחוני גדול , שביקש מכם להכניס שתי פונקציות למחלקה Profile.
בתור מרגלים ומרגלות, תצטרכו לממש שתי פונקציות משעשעות במיוחד, שמשנות את הסטטוס בפרופיל וגורמות לדברים להישמע אחרת ממה שהתכוונו...

משנה את כל המילים בסטטוס למילה אחת מסוימת.
 	
דוגמה:
נניח ויש לנו פרופיל שבעמוד שלו יש סטטוס:
 	
`Have you ever wondered why you can't taste your tongue?`
`That's crazy!!!`
 	
אם נקרא למתודה עם המילה "Magshimim" הסטטוס ישתנה ל:
 	
`Magshimim Magshimim Magshimim Magshimim`
`Magshimim Magshimim Magshimim Magshimim`
`Magshimim Magshimim`
`Magshimim Magshimim`
 	
שימו ❤️ , אם במחרוזת המקורית היו רווחים, טאבים או ירידות שורה הם צריכים להופיע גם בסטטוס החדש.
	`void changeAllWordsInStatus(std::string word)`

מחליפה את כל המופעים של מילה מסוימת במחרוזת, למילה אחרת.
 	
דוגמה:
נניח ויש פרופיל עם עמוד שבו הסטטוס הוא:
 	
`No matter how smart you are, you can never convince someone stupid that they are stupid.`
 	
אם נקרא למתודה כדי להחליף את המילה "you" עם המילה "MMMMMMMMM" הסטטוס ישתנה ל:
 	
`No matter how smart MMMMMMMMM are, MMMMMMMMM can never convince someone stupid that they are stupid.`
	`void changeWordInStatus(std::string word_to_replace,`
	`std::string new_word)`

את הפונקציות ניתן לבדוק ע"י הרצת הטסט test3Bonus שבקובץ *test3Profile.cpp*

שלב 6: רשת חברתית – מחלקת Social Network

מחלקת Social Network – רקע

וואו זה היה ארוך... אבל עכשיו סוף סוף יש לנו את כל החלקים ואנחנו יכולים להגדיר את המחלקה האחרונה והחשובה ביותר – SocialNetwork אשר מייצגת רשת חברתית.

מחלקת SocialNetwork

בחלק הזה ניצור 2 קבצים חדשים – *SocialNetwork.h* ו-*SocialNetwork.cpp*
בנוסף, בשלב הזה נשתמש ברשימה מקושרת שנכתבה ומומשה עבורנו בקבצים *ProfileList.h* ו-*ProfileList.cpp*.

המחלקה מייצגת רשת חברתית, שמחזיקה את שם הרשת, רשימה של פרופילים וגיל מינימלי שאסור שהמשתמשים שלה יהיו צעירים ממנו.

הגדירו את המחלקה וממשו אותה ע"פ התיאור הבא:

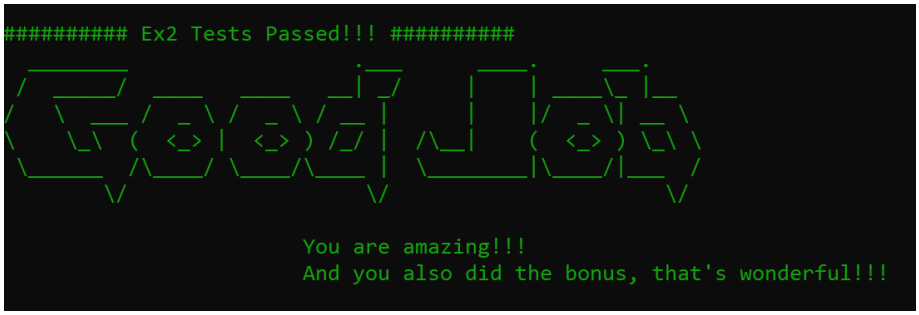
רשת חברתית – **Social Network** 	
תיאור	**מתודה/פעולה**
מאתחלת אובייקט של רשת חברתית. **הערה**: הפונקציה תיקרא לפני השימוש באובייקט.	`void init(std::string networkName, int minAge)`
מנקה את האובייקט ומוחקת זיכרון שהוקצה לאובייקט. **הערה**: אם במהלך יצירת האובייקט איתחלנו אובייקטים אחרים, אז בפונקציה זו אנו ננקה גם אותם. אולם אין צורך להשתמש ב-`delete` אלא אם כן הייתה הקצאת זיכרון דינמית באמצעות המילה `new` .	`void clear()`
מחזירה מחרוזת עם שם הרשת	`std::string getNetworkName();`
מחזירה את הגיל המינימלי שממנו אפשר להצטרף לרשת.	`int getMinAge();`
מוסיפה פרופיל חדש לרשימת הפרופילים של הרשת החברתית. **הערה**: אם הפרופיל בבעלות של משתמש צעיר מהגיל המינימלי המותר ברשת, הפונקציה צריכה להחזיר `false` ולא להוסיף את הפרופיל לרשת. **הערה נוספת:** הפונקציה צריכה לעשות שימוש במחלקה `ProfileList` שניתנה עם קבצי התרגיל.	`bool addProfile(Profile profile_to_add)`

רשת חברתית – **Social Network (המשך...)**	
תיאור	**מתודה/פעולה**
מחזירה מחרוזת עם רשימה של מכשירים שמריצים מערכת הפעלה Windows ומחוברים לרשת החברתית. עבור כל מכשיר יתווסף למחרוזת המספר המזהה שלו ומערכת ההפעלה שלו, אח"כ פסיק ואז רווח. דוגמה: `[2123, Windows11], [1121, Windows10], [5361, Windows7], [7711, Windows11], [9444, Windows11]` שימו ❤️, עבור כל מכשיר נכניס למחרוזת סוגריים מרובעים שבתוכן המספר המזהה של המכשיר, רווח ופסיק, ואת מערכת ההפעלה שלו.	`std::string getWindowsDevices()`

הנחיות והערות חשובות:

- בהוראות לא רשומות החתימות המדויקות, אבל האחריות שלכם/ן היא להוסיף `const` ולקבל פרמטרים כ-reference היכן שצריך.
- המחרוזת שחוזרת מהמתודה `getWindowsDevices()` לא מכילה פסיק `','` ורווח בסוף, חשבו איך לגרום לתווים לא להופיע.
- זכרו לבצע `#include "ProfileList.h"` כדי שתוכלו להשתמש ברשימה שסופקה לכם/ן.
- מומלץ לבדוק את המחלקה לפני שממשיכים לסעיף הבא, תוכלו להיעזר בקובץ ***Tester.cpp*** שניתן יחד עם קבצי התרגיל. בתוך ה-Tester ניתן להריץ את הבדיקה ***test4SocialNetwork()***.

שימו 💜, עבשיו כבר ניתן להריץ את ה-Tester במלואו, אם כל הטסטים עברו תקבלו את המסך הבא.

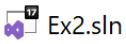


במידה והייתה בעיה עם אחד הטסטים, ה-Tester ידפיס הודעת שגיאה ויכוון למתודות שעליהן כדאי להסתכל.

נספחים

הגשה ב-GIT

- את הפרויקט יש לנהל ב-Git, לגשת אל ה- repository שהמדריכ\ה פתח\ה לנו ב github ולהגיש לינק לפרויקט ב-ב MOODLE (אפשר לעשות comment עם הלינק או להגיש מסמך txt עם הלינק בפנים).
- יש להעלות ל-repository את כל הקבצים הרלבנטיים לתרגיל (קבצי txt, מסמכים, ומשאבים אחרים שבהם השתמשנו).
חשוב להעלות את פרויקט ה-Visual Studio השלם ולהתעלם מקבצים לא נחוצים (הנחיות במסמך הבא), במידה ולא הועלה הפרויקט השלם, אין להעלות את שאר הקבצים שיוצר Visual Studio – הם רבים מאוד, הם לא מכילים מידע נחוץ להרצת הפרויקט אצל המדריך, ורק יוצרים בלגן.
- הבחירה אילו קבצים להעלות ל-repository נעשית באמצעות הפקודות add ו-rm. אופציה נוספת (מומלצת) היא להוסיף קובץ gitignore. אשר יתעלם מהקבצים הלא נחוצים.
- כשסיימתם/ן, בדקו שניתן להריץ את הפרויקט בקלות – בצעו Clone אל תיקייה במחשב אשר שונה מזו שעבדתם/ן, ותראו שהפרויקט נפתח ע"י לחיצה על קובץ ה-sln ויכול לרוץ בלי בעיה



דגשים:

- על המטלה צריך לעבוד ב repository שהמדריכ\ה פתח\ה לנו ב github.
- יש לוודא שכל הקבצים הרלבנטיים נוספו ל-repository (באמצעות הפקודה add), במידת הצורך ניתן להוריד קבצים מיותרים (באמצעות הפקודה rm)
- יש לבצע commit עבור כל סעיף, ובנקודות שבהן הוספנו שינויים חשובים (לפי הדגשים שהועברו בכיתה).
- עבור כל commit, זכרו לכתוב הודעה קצרה ואינפורמטיבית, שאפשר יהיה להבין מה היה השינוי בקוד.
- יש לדחוף את הקוד (באמצעות הפקודה push) ל-repository בסיום העבודה שלנו, חשוב שבסיום העבודה שלנו, ובמידה ונפנה למדריך/ה, ב-repository יהיה הקוד המעודכן ביותר.
- במידה ושכחנו או שאנחנו לא בטוחים איך מעלים קובץ, או מתעלמים מקבצים, כדאי לצפות בסרטון הבא., או לעיין בתרגולים הבסיסיים על גיט וגיטהב
- בסיום העבודה יש להגיש לכיתת ה-Moodle קישור ל-repository.
נקודה חשובה מאוד, לא ליצור תיקיות בשם ex1, ex2 ... ex13. הסיבה לכך היא ש-שרת הבדיקות מייצר תיקיות כאלה לכל תרגיל בהתאמה, ואם הן כבר יהיו קיימות בפרוייקט, הבדיקות לא ירוצו כלל. אפשר לייצר תיקיות עם שם דומה כמו HW1 וכו'
- **שימו למבנה התיקיות הראשוני של התרגיל, אם יש תתי-תיקיות לסעיפים שונים, תעבדו על אותם תתי-סעיפים בתיקיות האלו. – הבדיקות האוטומטיות מסתמכות על זה.**

כללי

1. יש לבדוק שכל **המטלות מתקמפלות ורצות ב-**VS2022. מטלה שלא תעבור קומפילציה אצל הבודק לא תיבדק **והניקוד שלה יהיה 0** 😵
2. יש לבדוק שהקוד שכתבתם עובד. יש להריץ בדיקות שלכם ולוודא שהקוד ברמה טובה.
3. כאשר אתם מתבקשים לממש פונקציה, ממשו בדיוק את הנדרש. אין להוסיף הדפסות וכדו'. אם הוספתם תוך כדי הבדיקות שלכם הדפסות, אנא דאגו להוריד אותם לפני ההגשה.
4. להזכירכם! העבודה היא עצמית, ואין לעשות אותה ביחד.
5. על כל שאלה או בעיה יש לפנות למדריך, לפחות 36 שעות לפני מועד ההגשה.

דגשים לתכנות נכון

- כדאי לקמפל כל מספר שורות קוד ולא לחכות לסוף! הרבה יותר קל לתקן כאשר אין הרבה שגיאות קומפילציה. בנוסף קל יותר להבין מאיפה השגיאות נובעות.
- כדאי לכתוב פונקציה ולבדוק אותה לפני שאתם ממשיכים לפונקציה הבאה. כלומר, כתבו תכנית ראשית שמשתמשת בפונקציה ובודקת האם היא עובדת כראוי. חישבו על מקרי קצה ונסו לראות מה קורה.
- בכל פעם שאתם מתקנים משהו, זכרו שיכול להיות שפגעתם במשהו אחר. לכן עליכם לבדוק שוב מהתחלה.
- חשפו החוצה רק את הממשק המינימלי הדרוש (minimal API), הגדירו את שדות המחלקה כפרטיים, וכמה שפחות מתודות כציבוריות.

בדיקות אוטומטיות באתר GitHub – תרגיל 2

לפני שממשיכים להסברים על הבדיקות של התרגיל הנוכחי,
מומלץ לעבור על המסמך הבא שמסביר על מערכת הבדיקות האוטומטיות באופן כללי:

✳ קישור להסבר על GitHub ומערכת הבדיקות ✳

ישנם כמה טסטים שצריך לעבור בתרגיל מספר 2:

- **Verify Submission**
  בודק שהגשנו את התרגיל בצורה נכונה:
  - יש קובץ .gitignore ב-repo
  - שבתוך ה-repo יש פרויקט VS
  - שאין קבצים מיותרים שמונעים פתיחה נוחה של הפרויקט (לדוגמה תיקיית x64 או תיקיית vs. או קובץ vcxproj.users שנוצרים ע"י VS).
    חשוב לזכור שאם הוספנו gitignore. אז הקבצים האלו לא יעלו ל-repo ואנחנו לא צריכים לעשות כלום.

- **Look for pdf file**
  בודק שהגשנו קובץ בשם **answers.pdf**.
  בתוך הקובץ הזה צריכות להיות רשומות התשובות לחלק התיאורטי (על reference).

- **בדיקות קומפילציה**
  בודקות שהחלקים השונים של התרגיל מתקמפלים ורצים בהצלחה
  כדי לוודא שהבדיקות יעברו, אפשר להוריד את הטסטרים מתיקיית ה-Google Drive של התרגיל (שאליה אפשר לגשת דרך ה-MOODLE).

  - **Part 1 tests + Compile Part 1**
    מקמפל את הקבצים Device.cpp, Device.h והטסטר שיש ב-drive (נקרא test1Device.cpp)
    אם הקומפילציה מצליחה, ירוץ טסט ויבדוק את מחלקת Device.

  - **Part 2 tests + Compile Part 2**
    מקמפל את הקבצים של מחלקת User ומחלקת Device עם הטסטר שיש ב-drive (נקרא test2User.cpp)
    אם הקומפילציה מצליחה, ירוץ טסט ויבדוק את מחלקת User.

  - **Part 3 tests + Compile Part 3**
    מקמפל את הקבצים של מחלקות Profile, User ו-Device עם הטסטר שיש ב-drive (נקרא test3Profile.cpp)
    אם הקומפילציה מצליחה, ירוץ טסט ויבדוק את מחלקות Profile ו-Page.

- **Part 4 tests + Compile Part 4**
  מקמפל את הקבצים של מחלקות SocialNetwork, Profile, User ו-Device עם הטסטר שיש ב-drive (נקרא Tester.cpp)
  אם הקומפילציה מצליחה, ירוץ טסט ויבדוק את המחלה Social Network.

- **Bonus tests + Compile Bonus**
  מקמפל את הקבצים של מחלקות Profile, User ו-Device, אם הקבצים התקמפלו, ירוץ טסט שיבדוק את המתודות של סעיף הבונוס (`changeWordInStatus` , `changeAllWordsInStatus`)

- **בדיקות זיכרון**
  הטסט בודק ששיחררנו את כל הזיכרון שהקצינו דינאמית.
  כלומר אם עשינו new אז הטסט מוודא שגם עשינו delete מתאים.

- **בדיקות כלליות**
  יהיו גם בדיקות כלליות שירוצו:

  - **Verify Necessary Exercise Files**
    בודק שהגשנו את כל הקבצים הנחוצים (כולל קבצי main שבהם השתמשנו לבדוק את הקוד של עצמנו).

  - **VerifySubmissionDate**
    בודק שהגשנו את התרגיל בזמן.
    חשוב לזכור שגם אם הגשנו את התרגיל אחרי הזמן והטסט נכשל, זה לא משפיע עלינו, וזה לא ימנע ממנו להעלות משהו נוסף.
    הבדיקה בסה"כ נועדה ליידע את המדריך והרכזת על הזמן שבו הגשנו את הקוד.

בסיום העבודה, ה-action צריך להיראות ככה:

